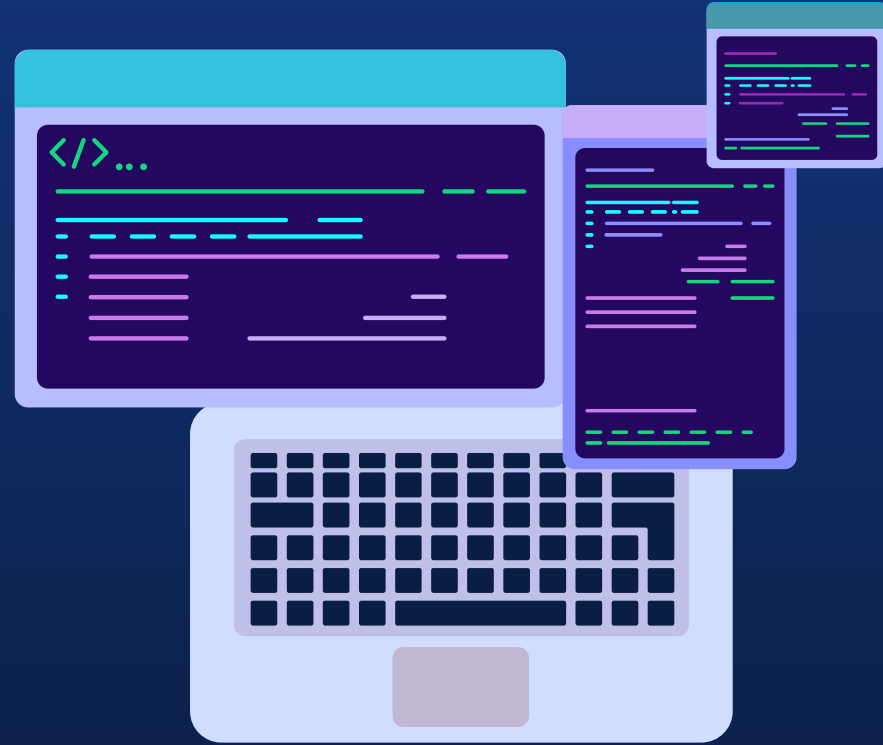


Java 0-0 Program/Visualizer Project

By Prottasha Deb, Angelina Martis,
Arya Patel & Aiden Stensrud





Goal & Motivation

- Problem: Novice coders sometime struggle with concepts of classes and objects.
- Solution: The purpose of this project intends to give students/novice coders:
 - Visualization of internal memory of a Java program
 - Animation of changes in memory
 - A better comprehension of how memory is allocated to different objects and classes and how they work in a program



Approach

The User can:

- enter a Java program into the visualizer.
- run the program and view its execution step-by-step.
- see how variables and objects are created in memory.
- see how object references connect variables to objects.
- observe how method calls and parameter passing affect memory.
- see how memory changes as each instruction executes.
- use animated memory diagrams to better understand what is happening behind their Java code.
- visualize the relationship between their Java code and the program's memory state.





Novel Features

A Java O-O Visualizer was previously attempted by a Senior Design group & they did not successfully focus on **accurate memory diagrams!**

Our project:

- Places memory visualization at the center of the system.
- Focuses on accurately representing Java program execution.
- Provides a clear and understandable visual representation of:
 - Program memory
 - Objects and references
 - Variables
 - Method execution





Algorithms and Tools

Lexical Analysis	• Break Java source code into meaningful tokens
Syntax Analysis	• Analyze the structure of the Java program • Determine how different parts of the code relate to each other
Memory Diagrams	• Represent the program's memory in a visual format





Technical Challenges

Limited compiler knowledge	● Team needs to research and learn how Java programs can be parsed and represented
Memory diagram implementation	● Convert parser output into the client's desired visual design
Animation complexity	● Accurately animate low-level program information
Complex programs	● Large programs can create crowded memory diagrams
Dynamic organization	● Need to automatically organize diagrams so information remains easy to understand



Milestone 1

- Compare/Research
 - Collaboration Tools for Software Development, Docs/Presentations, Communication, Task Calendar, etc
 - Optimal application to implement the visualizer (web or desktop)
 - Tools/Strategies for Java parser and memory instruction animation
- Resolve Technical Challenge: Lack of compiler knowledge
- Design/Demo first draft for the visualizer output memory diagrams
- Create a Requirement, Design, and Test Plan document

Milestone 2

- Implementation, Testing, and Demonstrations
 - Java Code Parser
 - Simple Memory Diagram Generator
 - Conceptualize and draft memory diagram animations
- Resolve Technical Challenge: Memory diagram design for complicated memory states

Milestone 3

- Implementation, Testing, and Demonstrations
 - Memory Diagram Animator
 - Memory Diagram Generator for Complex Memory States
 - Full Cohesive Application that combines the Parser, Generator, and Animator
 - UI for chosen type of Application



Task Matrix (Milestone 1)



Task	Arya	Aiden	Prottasha	Angelina
Resolve technical challenges	-	lack of compiler knowledge	-	-
Design memory diagram demo	design 50%	give input	give input	design 50%
Compare and select the application type	web	-	-	desktop
Compare and select Collaboration Tools	presentation	programs	documents	communication, task calendar
Compare and select technical tools/strategies	-	java code parser	memory instruction animator	-
Requirement Document	write 10%	write 30%	write 50%	write 10%
Design Document	write 10%	write 50%	write 30%	write 10%
Test Plan	write 10%	write 30%	write 50%	write 10%





THANKS!

Any Questions?

